



Desenvolvimento Web 1

Prof. Diego Max



Aula 11:

Operadores Aritméticos, Relacionais
e Lógicos



Aula 11.1:

Operadores Aritméticos e
de Atribuição

Operadores em JavaScript



Podemos dividir os operadores em JavaScript por categorias, algumas delas são:

aritméticos

atribuição

relacionais

lógicos

ternário

JS

Operadores Aritméticos



Soma



Subtração



Multiplicação



Divisão real



Resto da divisão inteira



Potência



JS

Operadores Aritméticos

Em uma operação com mais de um operador deve-se atentar que alguns operadores são resolvidos primeiro do que outros.

Exemplo:

$$3 + 4 / 2 = 3.5$$

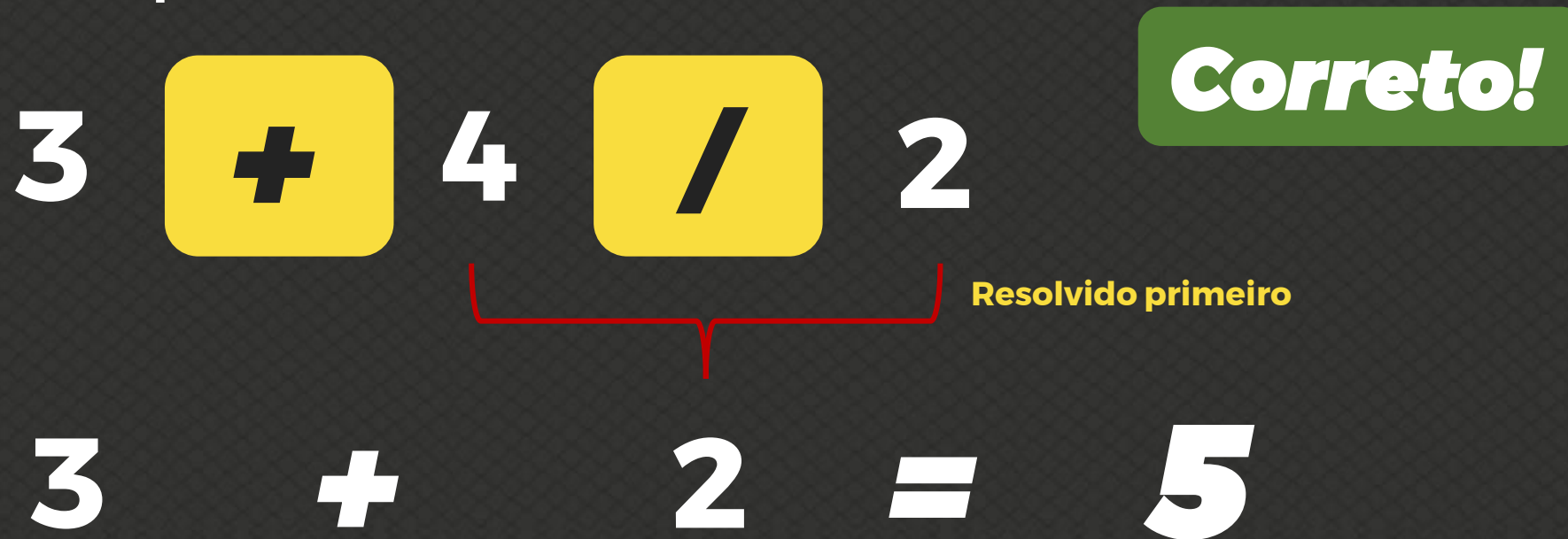
Errado!

JS

Operadores Aritméticos

Em uma operação com mais de um operador deve-se atentar que alguns operadores são resolvidos primeiro do que outros.

Exemplo:



3 + 4 / 2

Resolvido primeiro

3 + 2 = 5

Correto!

{ }

JS

Operadores Aritméticos

Para mudar a ordem das operações basta inserir parênteses. Cálculos entre parênteses serão executados primeiro:

$$(3 + 4) / 2 = 3.5$$

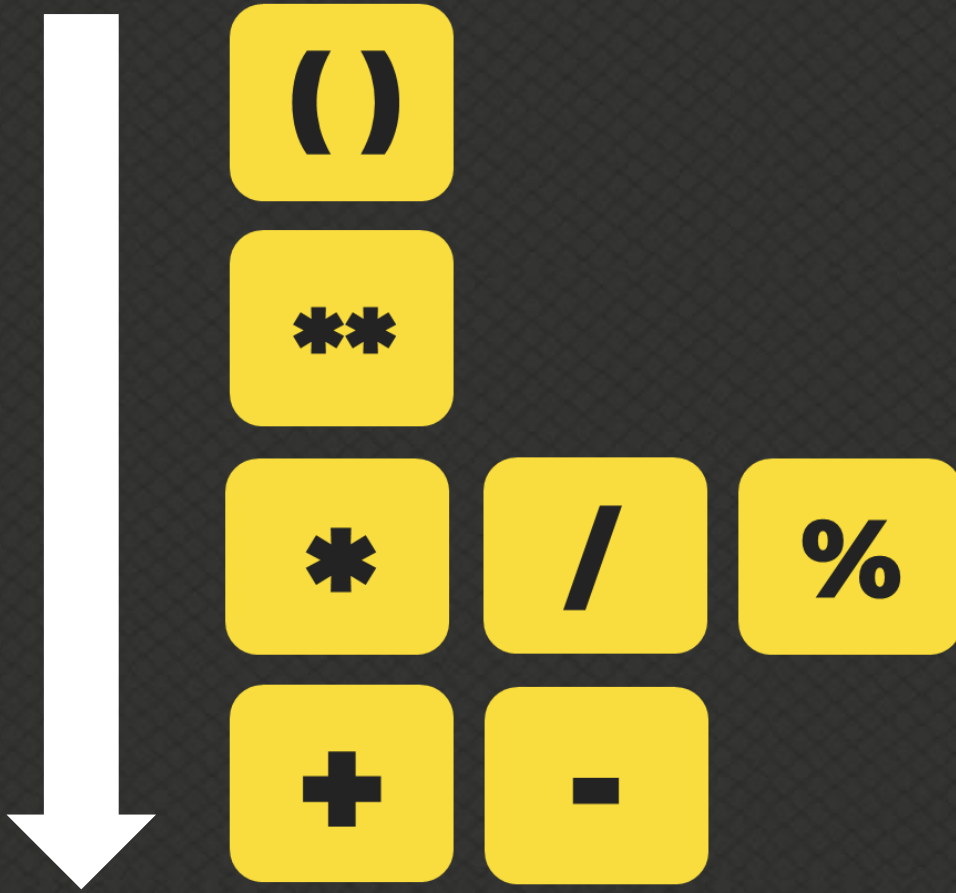


Resolvido primeiro



JS

Ordem de Precedência



`{ }`

JS

Ordem de Precedência



Dica:

Podemos lembrar essa ordem usando **PEMDAS**:
Parênteses, **E**xpoentes, **M**ultiplicação e **D**ivisão (da esquerda para a direita), **A**dição e **S**ubtração (da esquerda para a direita).

JS

Auto atribuições

Auto atribuição é uma atribuição a própria variável.

Exemplos:

var n = 3

Simplificando:

n = n + 4

n += 4

(**3** + 4 resultado n = **7**)

n = n - 5

n -= 5

(**7** - 5 resultado n = **2**)

n = n * 4

n *= 4

(**2** * 8 resultado n = **8**)

n = n / 2

n /= 2

(**8** / 2 resultado n = **4**)

n = n ** 2

n **= 2

(**4** ** 2 resultado n = **16**)

n = n % 4

n %= 4

(**16** % 4 resultado n = **0**)



JS

Incremento

Incrementar é um termo comum na programação, que se refere a adicionar **1** a uma variável, e armazenar o valor na própria variável.

Exemplos:

var x = 5

Simplificando:

x = x + 1

x ++ (**5** + 1 resultado **x = 6**)

x ++ (**6** + 1 resultado **x = 7**)

x ++ (**7** + 1 resultado **x = 8**)



JS

Decremento

Decremento se refere a subtrair **1** a uma variável, e armazenar o valor na própria variável.

Exemplos:

var x = 5

Simplificando:

x = x - 1

x -- (**5** - 1 resultado **x = 4**)

x -- (**4** - 1 resultado **x = 3**)

x -- (**3** - 1 resultado **x = 2**)



JS

Pré-incremento e pré-decremento



Chamamos de **pré-incremento** ou **pré-decremento** quando estes são declarados antes da variável. Quando declarados depois, chamamos de **pós-incremento** / **pós-decremento**. Veremos mais à frente a diferença concreta entre eles.

Pós-incremento: **x ++**

Pós-decremento: **x --**

Pré-incremento: **++ x**

Pré-decremento: **-- x**

JS



Aula 11.2:

Operadores Relacionais, Lógicos e Ternário

Operadores Relacionais

(ou de comparação)

Maior



Menor



Maior ou igual



Menor ou igual



Igual



Diferente



JS

Operadores Relacionais

Para toda expressão que tenha um **operador relacional** ligado a ela, o resultado dessa expressão será sempre **TRUE** (verdadeiro) ou **FALSE** (falso).

Exemplos:

5	>	2	→	TRUE
7	<	4	→	FALSE
8	>=	8	→	TRUE
9	<=	7	→	FALSE
5	=	5	→	TRUE
4	!=	4	→	FALSE

Ordem de Precedência



Na ordem de precedência os **operadores aritméticos** são resolvidos **primeiro**, depois os **operadores relacionais**.



Aritméticos

Relacionais

8 **<=** 15 **-** 10
→ **FALSE**

JS

Operadores Relacionais de Identidade



O **operador de igualdade restrita** “**===**” (ou idêntico) verifica se as variáveis possuem o mesmo valor e são do mesmo tipo.

5 **==** 5 → **TRUE**

5 **==** '5' → **TRUE**

5 **===** '5' → **FALSE**

5 **===** 5 → **TRUE**

JS

Operadores Relacionais de Identidade



O **operador desigual restrito** “**!==**” verifica se as variáveis ou não possuem o mesmo valor ou não são do mesmo tipo.

5 **!=** '5' → FALSE

5 **!==** '5' → TRUE

JS

Operadores Lógicos



Negação



Não (NOT)

Conjunção



E (AND)

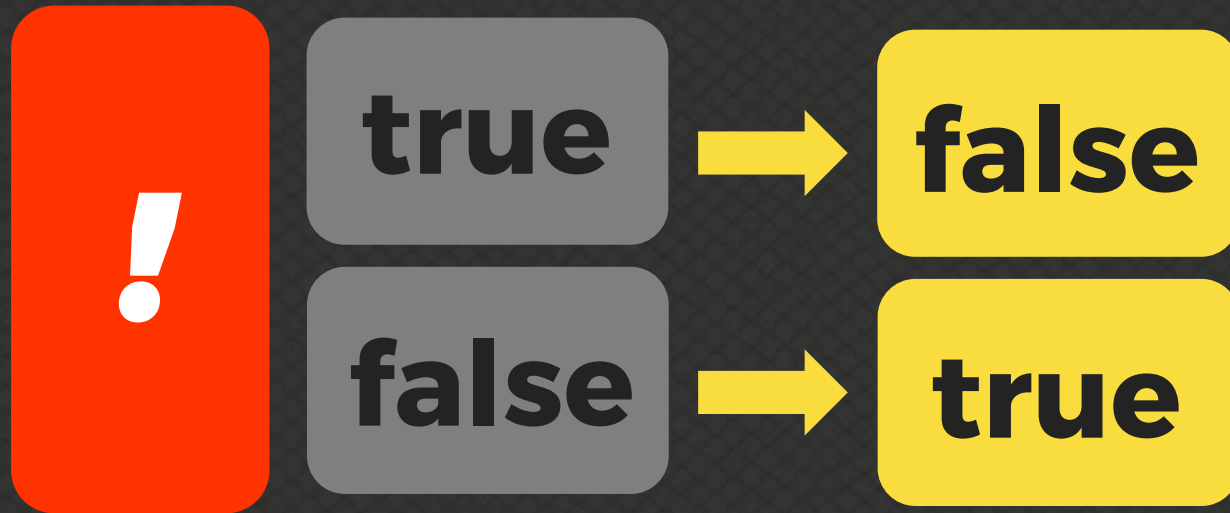
Disjunção



Ou (OR)

JS

Operador de Negação



```
if (! num ) {  
  alert("Você não digitou  
  um número.")  
} else {  
  alert("Você digitou um  
  número")  
}
```

JS

Operador de Conjunção

{ }

true	&&	true	→	true
true		false	→	false
false		true	→	false
false		false	→	false

JS

Operador de Conjunção



```
> var a = 5
undefined
> var b = 8
undefined
> a < b && b % 2 == 0
true
> a > b && b % 2 == 0
false
>
```

JS

Operador de Disjunção

{ }

true	//	true	→	true
true		false	→	true
false		true	→	true
false		false	→	false

JS

Operador de Disjunção



```
> var a = 5  
undefined  
> var b = 8  
undefined  
> a <= b || b / 2 == 2  
true  
> a >= b || b / 2 == 2  
false  
>
```

JS

Ordem de Precedência



Na ordem de precedência os **operadores aritméticos** são resolvidos primeiro, depois os **operadores relacionais** e por fim os **operadores lógicos**.



Aritméticos

`() , ** , / ...` (PEMDAS)

Relacionais

`> , < , >= ...`

Lógicos:

`! , && , ||` (nesta ordem)

JS

Operadores Ternário



teste



true



false

Média >= 7.0



“Aprovado”



“Reprovado”

JS

Operadores Ternário



```
> var media = 5.5  
undefined  
> media > 6 ? 'APROVADO' : 'REPROVADO'  
'REPROVADO'  
> media > 5 ? 'APROVADO' : 'REPROVADO'  
'APROVADO'  
>
```

JS

Operadores Ternário



```
> var idade = 19
undefined
> var r = idade >= 18 ? 'MAIOR' : 'MENOR'
undefined
> r
'MAIOR'
> _
```

JS

Operadores Ternário



```
> var x = 8
undefined
> var res = x % 2 == 0 ? 5 : 9
undefined
> res
5
>
```

JS



Desenvolvimento Web 1

Prof. Diego Max